LMJuly18-03 — LaunchMind: Backend

Date: July 18, 2026 · Part 3 of 6 **Runtime:** Node.js + Fastify **Host:** Oracle Cloud VM (Docker) **Entry point:** `backend/src/server.ts`

Table of Contents

- File Structure
- Server Bootstrap
- All Route Endpoints
- Core Lib Files
- Services
- Agents (12)
- Workers (5)
- Middleware
- Types
- Tests
- Key Patterns & Rules

1. File Structure

```
backend/
├── src/
│   ├── server.ts
│   ├── lib/
│   │   ├── aiClient.ts
│   │   ├── aiPlatform.ts           ← MANDATORY AI entry point
│   │   ├── contextEngine.ts
│   │   ├── promptRegistry.ts
│   │   ├── modelRouter.ts
│   │   ├── tokenVault.ts
│   │   ├── tokens.ts
│   │   ├── replicateClient.ts
│   │   ├── creatomateClient.ts
│   │   ├── elevenLabsClient.ts
│   │   ├── jwtPlugin.ts
│   │   ├── scheduler.ts
│   │   ├── scraperQueue.ts
│   │   ├── supabaseAdmin.ts
│   │   ├── response.ts
│   │   └── errorCodes.ts
│   ├── routes/                      ← 24 route files
│   ├── services/                    ← 27 service files
│   │   └── agents/                  ← 12 agent files
│   ├── workers/                     ← 5 worker files
│   ├── types/                       ← 5 type files
│   └── middleware/
│       └── auth.middleware.ts
├── migrations/                      ← 61 SQL files
├── tests/                           ← 20 test files
└── vitest.config.ts
```

2. Server Bootstrap (`server.ts`)

Startup sequence:

- Load `.env.local` (local dev only, no-op in production)
- Initialize Sentry (before any plugin — captures startup errors)
- Build Fastify instance
- Register `@fastify/cors` (origin: NEXT_PUBLIC_APP_URL + localhost:3000)
- Register `@fastify/rate-limit` (100 req/min per IP)
- Register `jwtPlugin` (Supabase JWKS, ES256)
- Register `InsufficientTokensError` handler
- Register all 24 route plugins

- Start BullMQ workers: `startBriefWorker()`, `startIntakeWorker()`, `startContentWorker()`, `startMissionWorker()` (all Redis-gated — no-op if Redis not configured)
- Schedule weekly brief cron: `scheduleWeeklyBrief()`
- `server.listen({ port: 3001, host: '0.0.0.0' })`

Exported function: `buildServer(): Promise<FastifyInstance>` (used in tests without port binding)

---

3. All Route Endpoints

products.route.ts

Method	Path	Description
POST	/products	Create product
GET	/products	List founder's products
GET	/products/:id	Get product by id
PUT	/products/:id	Update product
DELETE	/products/:id	Soft-delete product
POST	/products/scrape	Trigger scrape job
PUT	/products/:id/confirm	Confirm ICP + finalize product
POST	/products/intake/context	Save founder context (JSONB merge)
POST	/products/intake/screenshots	Upload screenshots
POST	/products/:id/generate-strategy	Trigger strategy generation
GET	/products/:id/strategy	Fetch generated strategy
POST	/products/:id/archive	Archive product
POST	/products/:id/restore	Restore archived product
POST	/products/setup/start	Intake v3: start setup wizard
PUT	/products/intake/step/:step	Save intake wizard step
POST	/products/intake/complete	Complete intake
GET	/products/intake/status	Poll intake job status

owner.route.ts

Method	Path	Description

GET	/owner/brief	Morning Brief aggregation (founder + product + AI rec + approvals + opps + timeline + memories)
GET	/owner/opportunities	Growth backlog
POST	/owner/opportunities	Create opportunity
PATCH	/owner/opportunities/:id	Update state (save/dismiss/convert)
POST	/owner/ask	Ask LaunchMind (Context Engine + Sonnet)
GET	/owner/results	Interpreted campaign metrics
GET	/owner/timeline	Mission + campaign event stream
GET	/owner/notifications	Notifications + unread count
PATCH	/owner/notifications/:id/read	Mark notification read

campaigns.route.ts

Method	Path	Description
POST	/campaigns/create	Create campaign
GET	/campaigns/:id/detail	Campaign detail + metrics + approvals
PUT	/campaigns/:id	Update campaign
POST	/campaigns/:id/plan	Generate campaign plan (Haiku)
POST	/campaigns/:id/schedule	Schedule campaign (§1.5 enforced)
POST	/campaigns/:id/launch	Launch campaign (§1.5 + §1.6 enforced)
POST	/campaigns/:id/resume	Resume paused campaign
POST	/campaigns/:id/cancel	Cancel campaign
POST	/campaigns/:id/archive	Archive campaign
POST	/campaigns/:id/assets	Link content assets to campaign

experiments.route.ts

Method	Path	Description
POST	/experiments	Create experiment
GET	/experiments	List experiments
GET	/experiments/:id	Get experiment + variants

POST	/experiments/:id/start	Start experiment
POST	/experiments/:id/results	Update variant results
POST	/experiments/:id/winner	Select winner → ingestLearningEvent
POST	/experiments/:id/archive	Archive experiment

calendar.route.ts

Method	Path	Description
GET	/calendar	Merged calendar (authored + campaign + experiment + brief events)
POST	/calendar	Create authored event
PUT	/calendar/:id	Update authored event
DELETE	/calendar/:id	Delete authored event

studio.route.ts

Method	Path	Description
POST	/studio/generate	On-demand asset generation (31 types)
GET	/studio/assets	List assets (search + filter + pagination)
GET	/studio/assets/:id	Asset + versionCount + publishTargets
PUT	/studio/assets/:id	Update (creates version snapshot first)
POST	/studio/assets/:id/transform	7 AI transforms (rewrite/tone/shorten/expand/localize/translate/ab_variant)
GET	/studio/assets/:id/versions	Version history
POST	/studio/assets/:id/archive	Soft delete
POST	/studio/assets/:id/restore	Restore archived
POST	/studio/assets/:id/publish	Publish (§1.5: 422 if approved_at is null)
GET	/studio/stats	Aggregate counts + byType + byStatus

missions.route.ts

Method	Path	Description
POST	/missions	Create mission
GET	/missions	List missions (with status filter)

GET	/missions/approvals	Pending approvals
GET	/missions/:id	Mission + steps
GET	/missions/:id/timeline	Mission event timeline
GET	/missions/:id/logs	Mission execution logs
POST	/missions/:id/cancel	Cancel mission
POST	/missions/:id/retry	Retry failed mission
POST	/missions/:id/approvals/:stepId	Respond to approval (approve/reject)

memory.route.ts

Method	Path	Description
GET	/memory	List memories (filter by type/product)
POST	/memory	Create memory
GET	/memory/:id	Get memory + versions
PUT	/memory/:id	Update memory (creates version)
DELETE	/memory/:id	Archive memory
POST	/memory/search	Semantic search
POST	/memory/events	Ingest learning event
POST	/memory/merge	Merge duplicate memories

knowledge.route.ts

Method	Path	Description
GET	/knowledge/graph	Full graph (nodes + edges)
GET	/knowledge/nodes/:id	Get node
POST	/knowledge/nodes	Create/upsert node
POST	/knowledge/edges	Create edge (owner-verified both nodes)
DELETE	/knowledge/nodes/:id	Delete node + edges
DELETE	/knowledge/edges/:id	Delete edge
POST	/knowledge/merge	Merge nodes

ai.route.ts

Method	Path	Description
GET	/ai/context/:productId	Build + return context package
GET	/ai/prompts	List prompts
GET	/ai/prompts/:promptId/versions	Prompt version history
POST	/ai/prompts	Register prompt (Studio-only, checkPlanFeature)
GET	/ai/audit	Paginated AI request audit (filter by status/promptId)
GET	/ai/audit/stats	Aggregated stats (requests, tokens, cost, success rate)

analytics.route.ts

Method	Path	Description
GET	/analytics/summary	Cross-product KPI totals
GET	/analytics/kpi	Weekly install series
GET	/analytics/attribution	Last-touch attribution by channel
GET	/analytics/funnel	Impressions → clicks → installs funnel
GET	/analytics/roi	ROI by channel (spend proxy + revenue proxy)
POST	/analytics/optimize	Generate optimization insights (Haiku)
GET	/analytics/insights	List optimization insights
PATCH	/analytics/insights/:id	Update insight status (applied/dismissed)

reports.route.ts

Method	Path	Description
GET	/reports	List reports (filter by type/productId)
POST	/reports/generate	Generate report (cache-first)
GET	/reports/:id	Get report content
GET	/reports/:id/export	Export as JSON
POST	/reports/:id/feedback	1–5 star feedback

recommendations.route.ts

Method	Path	Description
GET	/recommendations	List active recommendations
POST	/recommendations/generate	Generate (Builder+ gate via checkPlanFeature)
PATCH	/recommendations/:id/dismiss	Dismiss
PATCH	/recommendations/:id/save	Save
POST	/recommendations/:id/convert	Convert to mission
GET	/recommendations/history	All recommendations
POST	/recommendations/:id/feedback	Feedback

benchmarks.route.ts

Method	Path	Description
GET	/benchmarks	Category benchmarks (min 3 signals required)
GET	/benchmarks/categories	Available categories (cohort ≥3 filter)
GET	/benchmarks/trends	30/90 day trends
GET	/benchmarks/summary	Per-product cross-reference

Other Routes

Route file	Key endpoints
billing.route.ts	POST /billing/subscribe (Stripe/Razorpay), GET /billing/status, POST /billing/token-topup, POST /billing/webhook
channels.route.ts	Platform token CRUD + GA4/Firebase/website integrations
founders.route.ts	DELETE /founders/me (GDPR), GET /founders/me/export, GET /founders/me/sessions, PUT /founders/me/notifications
workspaces.route.ts	Workspace CRUD + member invite/remove + activate
settings.route.ts	GET/PUT /settings/content-preferences, POST /settings/voice-clone
contentAssets.route.ts	GET/POST /products/:id/content-assets
waitlist.route.ts	POST /waitlist
apiKeys.route.ts	Studio-only API key CRUD

admin.route.ts	GET /admin/stats, GET /admin/mrr

4. Core Lib Files

`aiPlatform.ts` — Mandatory AI Entry Point

```
// All services must import from here, never from aiClient.ts directly

export async function callSonnet(

  system: string,
  user: string,
  maxTokens: number,
  auditCtx: AuditContext,          // REQUIRED — not optional
  schema?: z.ZodSchema,            // optional Zod schema for output validation
): Promise<string>

export async function callHaiku(

  prompt: string,
  maxTokens: number,
  auditCtx: AuditContext,          // REQUIRED — not optional
): Promise<string>

export async function generateAI(opts: GenerateAIOptions): Promise<AIResponse>

// Full pipeline: context → resolvePrompt → routeModel → callSonnet/callHaiku → audit

type AuditContext = {

  founderId?: string | null;
  productId?: string | null;
  promptId: string;       // required
  action: string;         // required
}
```

Retry behaviour: 2 retries, 500ms → 1000ms backoff. Timeout: 60s (Sonnet) / 30s (Haiku). **Parse failure handling**: When model returns 200 but Zod schema validation fails, throws `OutputValidationError` and writes `status='failed'`, `error_message='output_validation_failed'` to `ai_requests`. **Prompt injection defense**: `sanitizeInput()` strips role markers (`system:` , `user:` , `assistant:`) and common instruction overrides (`ignore previous` , `new instruction` , etc.) before sending to the model. **Cost tracking**:

```
Sonnet: $3.00 / M input tokens + $15.00 / M output tokens
Haiku:  $0.25 / M input tokens + $1.25  / M output tokens
```

contextEngine.ts

```
export async function buildContextPackage(
  founderId: string,
  productId: string,
  opts?: { includeMemories?: boolean; includeKnowledgeGraph?: boolean; maxMemories?: number }
): Promise<ContextPackage>
```

Assembles from 6 parallel sources (all non-fatal — one failure doesn't break the package):

- Product details (confirmed_icp, brand_voice_profile, markets)
- Active campaigns + recent metrics
- Marketing memories (top-N by confidence)
- Knowledge graph (depth-1 traversal)
- Analytics summary (totalInstalls, topChannel)
- Recent learning events

```
export function formatContextForPrompt(pkg: ContextPackage): string
// Returns a readable string summary for insertion into prompts
```

modelRouter.ts

```
// ROUTING_TABLE — Sonnet for complex generation, Haiku for classification/scoring
const ROUTING_TABLE = {
  // Sonnet (3 actions)
  strategy_generation:       { model: 'sonnet', maxTokens: 8192 },
  morning_brief:             { model: 'sonnet', maxTokens: 512 },
  content_assets_generation: { model: 'sonnet', maxTokens: 12000 },
  // Haiku (8 actions)
  review_analysis:           { model: 'haiku', maxTokens: 1024 },
  icp_structuring:           { model: 'haiku', maxTokens: 2048 },
  brand_voice_extract:       { model: 'haiku', maxTokens: 400 },
  brand_voice_apply:         { model: 'haiku', maxTokens: 300 },
  content_score:             { model: 'haiku', maxTokens: 512 },
  recommendation_generation: { model: 'haiku', maxTokens: 2048 },
  agent_campaign_draft:      { model: 'haiku', maxTokens: 1024 },
  weekly_brief:              { model: 'haiku', maxTokens: 2048 },
  // Fallback
  default:                   { model: 'haiku', maxTokens: 1024 },
}
```

`promptRegistry.ts`

```
export async function resolvePrompt(name: string): Promise<Prompt | null>
export async function registerPrompt(name: string, body: string, model: string, maxTokens: number):
// Auto-increments version, archives previous active version
export async function listPrompts(): Promise<Prompt[]>
export async function listPromptVersions(name: string): Promise<Prompt[]>
```

`tokenVault.ts`

```
export async function encryptToken(plaintext: string): Promise<{ encryptedToken: string; kmsKeyId:
export async function decryptToken(encryptedToken: string, kmsKeyId: string, founderId: string): Pr
// Always writes to audit_logs before returning
// Never logs, never returns to frontend, never caches the decrypted value
```

`tokens.ts`

```
export async function consumeTokens(
  founderId: string,
  action: string,
  estimatedCost: number
): Promise<void>
// Phases 1–4: no-op (logs only). Phase 5+: deducts from founders.token_balance.
// Throws InsufficientTokensError if balance < estimatedCost (Phase 5+).
```

`replicateClient.ts`

Image generation via Flux.1 Schnell:

```
export type ImageStyle = 'photorealistic' 'graphic' 'mockup'

export async function generateImage(

  brief: string,
  style: ImageStyle,
  options?: { logoUrl?: string; brandColors?: string[] }
): Promise<string>  // returns CDN URL

// Style system:

// 'mockup' + marketingImages present  → use real screenshot + logo composite (no Flux.1)
// 'mockup' + no marketingImages       → Flux.1 photorealistic fallback
// 'photorealistic' + any              → Flux.1 with optional screenshot context hint
// 'graphic' + any                     → Flux.1 graphic/illustration style
```

Anti-split negative prompts: `ANTI_SPLIT` (no split-screen), `ANTI_TEXT` (no text/logos in image), `ANTI_DARK` (no dark/cinematic lighting). Emotion→lighting map forces warm/bright colors.

`supabaseAdmin.ts`

```
export function getSupabaseAdmin(): SupabaseClient
// Returns service-role client. Never expose to frontend.
// Used in all route handlers and services.
```

`response.ts`

```
export function ok<T>(data: T): { success: true; data: T }
export function fail(code: string, message: string): { success: false; error: { code, message } }
// Standard response envelope used by all routes
```

`jwtPlugin.ts`

```
// Uses supabase.auth.getUser() — algorithm-agnostic (handles both HS256 and ES256)
// Supabase rotated to ES256 on 2026-05-16
// request.jwtVerify() decorates request with founderId
```

---

5. Services

`strategyService.ts`

Generates 30/60/90 day strategy via Claude Sonnet. Uses `buildContextPackage()` for context. Fires `generateContentAssets()` fire-and-forget after strategy. Stores in `products.full_strategy` JSONB.

`contentService.ts`

6-step content generation pipeline:

- Build context from product + founder_context + ICP
- `callSonnet()` → structured JSON with all 31 asset types
- `callHaiku()` → enforce character limits + quality scoring
- ElevenLabs voice note (for voice_note assets)
- Creatomate video render (for video assets)
- Insert into `content_assets` table

Image generation decision tree:

```
style=mockup  + marketingImages[] → real screenshot + logo composite (no Flux.1 cost)
style=mockup  + no images         → Flux.1 photorealistic fallback
style=photo   + any               → Flux.1 with optional screenshot context hint
style=graphic + any               → Flux.1 graphic/illustration style
```

`icpService.ts`

- Cheerio scraper for App Store metadata (title, description, screenshots, ratings)
- Playwright scraper for Play Store + reviews (sandboxed worker)
- Logo extraction from website: checks apple-touch-icon → icon[type=png] → og:image → resolves relative→absolute URL → stored in `products.website_meta.logoUrl`
- `callMessages('haiku')` for screenshot multimodal analysis
- Returns `ScrapedAppData` (typed, Zod-validated)

`brandVoiceService.ts`

```
export async function extractBrandVoice(productId: string, founderId: string): Promise<BrandVoice>
// callHaiku with auditCtx: { founderId, productId, promptId: 'brand_voice_extract', action: 'brand

export async function applyBrandVoice(text: string, founderId: string, brandVoice: BrandVoice): Pro

// callHaiku with auditCtx: { founderId, productId: null, promptId: 'brand_voice_apply', action: 'b
```

`briefService.ts`

Weekly brief generation using Haiku. Called by `weeklyBriefWorker` every Sunday. Competitor re-scrape (App Store, Cheerio, diff against previous) included in brief pipeline.

`marketingImagesService.ts`

Permanent image collection at intake time. 3 sources (all best-effort):

- App Store / Play Store screenshots (up to 5) → downloaded to Supabase Storage
- Website hero/feature/banner images → downloaded to Supabase Storage
- Google Custom Search Images ("appName mobile app") → downloaded to Supabase Storage

Storage path: `content-assets/{founderId}/{productId}/source-images/`

Returns permanent Supabase Storage public URLs (CDN URLs from stores expire). Results stored in `products.scraped_meta.marketingImages[]`.

`marketingMemoryService.ts`

```
export async function createMemory(founderId, productId, data): Promise<MarketingMemory>
export async function updateMemory(id, updates): Promise<MarketingMemory>  // creates version recor
export async function archiveMemory(id): Promise<void>
export async function listMemories(founderId, filters): Promise<MarketingMemory[]>
export async function searchMemories(founderId, query): Promise<MarketingMemory[]>
export async function findDuplicateMemory(founderId, title, type): Promise<string | null>
export async function mergeMemories(sourceId, targetId): Promise<MarketingMemory>
export async function addEvidence(memoryId, content, source): Promise<void>
```

`knowledgeGraphService.ts`

```
export async function createNode(founderId, nodeType, label, properties): Promise<KnowledgeNode>  /
export async function createEdge(founderId, sourceId, targetId, relationType): Promise<KnowledgeEdg
// Verifies both nodes are owned by the same founder before creating edge

export async function getGraph(founderId): Promise<KnowledgeGraph>  // depth-1 traversal

export async function deleteNode(id, founderId): Promise<void>
export async function deleteEdge(id, founderId): Promise<void>
export async function mergeNodes(sourceId, targetId, founderId): Promise<KnowledgeNode>
```

`learningPipelineService.ts`

Single entry point: `ingestLearningEvent(founderId, eventType, payload)`. 8 event handlers:

Event type	Action
`intake_completed`	Creates product/brand/customer memories + knowledge nodes
`campaign_result`	Creates campaign performance memory
`review_ingested`	Creates customer insight memory
`founder_feedback`	Creates founder preference memory
`growth_brain_approved`	Updates product memory confidence
`analytics_synced`	Creates channel performance signal
`experiment_result`	Creates experiment learnings memory
`ai_conversation`	Creates interaction memory

missionService.ts

```
export async function createMission(founderId, productId, type, title, context, idempotencyKey?): P
// Idempotency-safe: returns existing active mission if idempotencyKey matches

export async function queueMission(missionId): Promise<void>   // draft → queued

export async function startMission(missionId): Promise<void>   // queued → running
export async function startStep(stepId): Promise<void>
export async function completeStep(stepId, output): Promise<void>
export async function failStep(stepId, error): Promise<void>
export async function requestApproval(missionId, stepId): Promise<void>  // → requires_approval
export async function respondToApproval(approvalId, status, note?): Promise<void>
// approved → step completed + re-queued | rejected → mission cancelled

export async function completeMission(missionId, result): Promise<void>

export async function failMission(missionId, error): Promise<void>
export async function cancelMission(missionId): Promise<void>
export async function retryMission(missionId): Promise<MissionJobPayload>  // failed → queued

export async function getNextPendingStep(missionId): Promise<MissionStep | null>

export async function getMission(missionId, founderId): Promise<Mission>
export async function listMissions(founderId, filters): Promise<Mission[]>
export async function getMissionSteps(missionId): Promise<MissionStep[]>
export async function getMissionLogs(missionId): Promise<MissionLog[]>
export async function getPendingApprovals(founderId): Promise<MissionApproval[]>
export async function logMission(missionId, founderId, message, level, metadata?): Promise<void>
```

decisionEngineService.ts

8 pure-TypeScript rule functions. Zero AI calls. AI cannot override these.

```
export class DecisionError extends Error {
  constructor(public statusCode: number, public code: string, public detail: string) { ... }
}

export function checkApprovalGate(campaign): void           // throws if approved_at is null

export function checkSpendCap(campaign, proposedBudget): void  // throws if over cap
export function checkPlanFeature(plan, feature): void       // throws if plan doesn't include featu
export function checkTokenBalance(balance, required): void  // throws if insufficient
export function checkRegenLimit(count, limit): void         // throws if over regen limit (sync)
export function checkExperimentRuntime(startDate, minHours?): void
export function checkWorkspacePermission(member, requiredRole): void
export function checkBenchmarkAccess(founderId): void       // no-op (all founders access)
```

`recommendationEngineService.ts`

Scoring formula: `score = impact×0.4 + confidence×0.3 + urgency×0.2 + source×0.1`

```
export async function generateRecommendations(founderId, productId): Promise<Recommendation[]>
// callHaiku → Zod-validated JSON → deduplicates by title → 14-day expiry → upserts to saved_opport

export async function expireStaleRecommendations(founderId): Promise<void>

// Sets state='dismissed' on recommendations past expires_at
```

`analyticsService.ts`

```
export async function getAnalyticsSummary(founderId): Promise<AnalyticsSummary>
export async function getKPITrend(founderId, productId, weeks?): Promise<KPIPoint[]>
export async function getAttribution(founderId): Promise<AttributionResult>  // last-touch by chann
export async function getFunnel(founderId): Promise<FunnelResult>
export async function getROI(founderId): Promise<ROIResult>
// spend proxy = CPI × installs; revenue proxy = ROAS × spend
```

`reportingService.ts`

Cache-first: checks `reports` table before calling AI.

- `weekly` + `experiment` reports → Haiku
- `monthly` + `executive` reports → Sonnet
- After weekly report → `ingestLearningEvent('founder_feedback')`

---

6. Agents (12)

All agents implement `AgentFn = async (input: unknown, context: AgentContext) => Promise<unknown>`.

`AgentContext` provides: `contextPkg`, `founderId`, `productId`, `log(message, level, metadata?)`.

Agent	Status	Key behaviour
`researchAgent`	Full	Scrapes + analyses product category + competitor data
`strategyAgent`	Full	Generates 30/60/90 day strategy via Sonnet
`contentAgent`	Full	Generates content asset batch via contentService
`campaignAgent`	Full	Drafts campaign configs; enforces §1.6 spend-cap per draft
`memoryAgent`	Full	Creates/updates marketing memories via marketingMemoryService

`reportingAgent`	Full	Generates weekly/monthly reports
`planningAgent`	Stub	Returns placeholder — M13
`creativeAgent`	Stub	Returns placeholder — M13
`publishingAgent`	Stub	Returns placeholder; enforces §1.5 (checks approved_at) — M13
`optimizationAgent`	Stub	Returns placeholder — M13
`learningAgent`	Stub	Returns placeholder — M13
`benchmarkAgent`	Stub	Returns placeholder — M13

`agentRegistry.ts` exports `AGENT_REGISTRY: Record<MissionType, AgentFn>` dispatch table.

---

7. Workers (5)

`intakeWorker.ts`

BullMQ consumer for `intake-queue`. Steps:

- Fetch product from DB
- Scrape App Store (Cheerio) or Play Store (Playwright via scraperWorker)
- Structure ICP via `icpService` (Haiku multimodal)
- Extract brand voice via `brandVoiceService`
- At 75% progress: `collectMarketingImages()` → stores permanent URLs in `products.scraped_meta.marketingImages[]`
- Update product with all scraped data
- Emit progress events (polled by `/products/intake/status`)

`weeklyBriefWorker.ts`

BullMQ consumer + Sunday cron. Each founder with active products:

- Re-scrape competitors (App Store, Cheerio)
- Diff competitor metadata against previous
- Generate brief via `briefService` (Haiku)
- Send via Resend
- Update `weekly_briefs` table

`contentWorker.ts`

BullMQ consumer for `content-queue`. Async content generation fired by `strategyService`. Calls `contentService.generateContentAssets()`.

`missionWorker.ts`

```
export async function startMissionWorker(): Promise<void>
export async function enqueueMission(payload: MissionJobPayload): Promise<void>
export async function stopMissionWorker(): Promise<void>
```

BullMQ consumer. Concurrency=5. Priority from `MISSION_PRIORITY`. DLQ via `missions.status='failed'`. Each job: fetch next pending step → dispatch to `AGENT_REGISTRY[step.agent_type]` → complete/fail step → check for next step or complete mission.

`scraperWorker.ts`

Playwright-based dynamic scraper. Runs in separate Docker container (`Dockerfile.scraper`). Sandboxed — no network access except to target stores.

---

8. Middleware

`auth.middleware.ts`

```
export function checkAnomaly(request: FastifyRequest): Promise<void>
// Called in foundersRoutes. Detects new device fingerprint or new country.
// Triggers: re-auth requirement + Resend alert email + audit_logs entry
// Redis-backed device tracking per founderId

export function extractFounderIdFromHeader(request: FastifyRequest): string | null

// Helper to extract founderId from Authorization header before jwtVerify
```

---

9. Types

`types/mission.ts`

Key types: `MissionType` enum (12 values), `MissionStatus` enum, `StepStatus` enum, `MissionPriority` enum, `AgentFn` type, `AgentContext` interface, `MissionJobPayload` interface. All Zod schemas for API validation.

`types/memory.ts`

Key types: `MEMORY_TYPES` array, `MEMORY_SOURCES` array, `MarketingMemory` interface, `KnowledgeNode` interface, `KnowledgeEdge` interface, `LearningEvent` interface. Zod schemas for all.

`types/scraper.ts`

`ScrapedAppDataSchema` (Zod): includes `marketingImages: z.array(z.string().url()).optional()`.
`WebsiteMetaSchema`: includes `heroImages: z.array(z.string()).optional()`, `logoUrl: string?`.

`types/strategy.ts`

Strategy output types: `StrategyOutput`, `ChannelStrategy`, `ContentCalendar`, `PlaybookInsight`.

`types/errors.ts`

`InsufficientTokensError` extends `Error`. Has `statusCode: 402` and `founderId`.

---

10. Tests

Test file	Tests	Status
aiPlatform.test.ts	25	24 passing (1 pre-existing fake-timers failure)
analytics.test.ts	16	16 passing
billing.test.ts	varies	passing
brief.test.ts	varies	passing
campaigns.test.ts	10	10 passing
channels.test.ts	varies	passing
content.test.ts	varies	1 pre-existing mock shape failure
experiments.test.ts	13	13 passing
health.test.ts	varies	passing
intake.test.ts	varies	passing
memory.test.ts	17	17 passing
metrics.test.ts	varies	passing
missions.test.ts	17	17 passing
owner.test.ts	19	19 passing
products.test.ts	varies	passing
recommendations.test.ts	28	28 passing
reports.test.ts	13	13 passing

strategy.test.ts	varies	passing
studio.test.ts	22	22 passing
waitlist.test.ts	varies	passing
workspace.test.ts	varies	passing

Total: 349/351 passing. 2 pre-existing non-blocking failures (unchanged since M09).

Run tests: `cd backend && npx vitest run`

---

11. Key Patterns & Rules

Route Handler Pattern

```
server.get('/owner/brief', async (request: FastifyRequest, reply: FastifyReply) => {
  await request.jwtVerify();        // always first
  try {
    const founderId = getFounderId(request);
    const supabase = getSupabaseAdmin();
    // ... business logic
    reply.send({ ... });
  } catch (err) {
    Sentry.captureException(err);
    reply.status(500).send({ error: (err as Error).message });
  }
});
```

Zod Validation Pattern

```
// CORRECT: safeParse inside handler
const body = MySchema.parse(request.body);
// or for graceful errors:
const result = MySchema.safeParse(request.body);
if (!result.success) return reply.status(400).send(fail('VALIDATION_ERROR', result.error.message));


// WRONG: never use Zod in Fastify schema: field


// schema: { body: MyZodSchema }  // ← this breaks
```

auditCtx Pattern (REQUIRED on all AI calls)

```
const auditCtx = {
  founderId,                          // string null undefined
  productId: product?.id ?? null,     // string null undefined
  promptId: 'action_name',            // required string
  action: 'action_name',              // required string (usually matches promptId)
};
const result = await callSonnet(SYSTEM_PROMPT, userPrompt, maxTokens, auditCtx);
```

vi.mock Hoisting Pattern (tests)

```
// CORRECT: all fixture data inlined inside factory function
vi.mock('../../lib/supabaseAdmin', () => ({
  getSupabaseAdmin: () => ({
    from: (table: string) => ({
      select: () => ({ data: [{ id: 'abc', title: 'Test' }], error: null }),
    }),
  }),
}));
// WRONG: referencing outer variables in vi.mock factory (hoisting breaks this)
```

Continue to: LMJuly18-04-Frontend.md